Hot and Cold Fields: Data Layout, Locality, and the Pointer-Chasing Problem

Course: High Performance Computing 1 (DSAI + SDIC)
Lecturer: Luca Tornatore
Academic Year: 2025/2026

---

1. The question we are asking

By this point in the course we have established, in some detail, how the cache hierarchy works: data travels between DRAM and the CPU in chunks of 64 bytes called *cache lines* (the figure may vary on some CPUs), a miss that reaches DRAM costs something in the range of 50–100 ns — or even more.
We had not yet discussed the hardware prefetcher (see in the slides, in the section about prefetching), but let's assume that it is "something" that is actually quite good at hiding that latency, provided the access pattern is regular enough for it to recognise.
Sequential access, fixed strides: fine.
Random pointer chasing: the prefetcher gives up completely, and you pay the full miss penalty at every step. Bad.

Given all of this, a natural question is: what can we do, specifically at the level of how we *design our data structures*, to make the cache hierarchy work in our favour? Not by changing the algorithm, not by restructuring loops — just by being more thoughtful about how the fields of a `struct` are laid out in memory.

The principle of locality helps us: if you know (and if you do not, unveil it) which fields of a struct will be needed at the same time, keep them close together in memory, ideally within the same 64 bytes, so that a single cache miss brings them all. Fields that are only touched rarely — once at the very end of a search, say — should be kept away from that hot path, so they do not waste cache lines that could otherwise hold useful data.

This is what is usually called *hot and cold field separation*. The concept is simple. What is less obvious — and this is where the interesting part is — is that doing the right thing in terms of memory layout does not always produce a timing improvement you can actually measure. Understanding why, and understanding what you should measure instead, is arguably the more valuable lesson here.

---

2. A running example: linked-list search

Consider a linked list with nodes defined as follows:

```
typedef struct node_t {
    double         key;         //  8 bytes, offset   0
    double         data[100];   // 800 bytes, offsets  8 - 807
    struct node_t *next;        //  8 bytes, offset 808
} node;                         // 816 bytes total
```

(We use `double data[100]` — 800 bytes — rather than `char data[300]` as in the slides).
One node is 816 bytes, which spans 13 cache lines (⌈816/64⌉ = 13).

Now suppose we want to search this list for the unique node whose `key` matches a given target value. The typical traversal is:

```
while ( (p != NULL) && (p->key != key) )
    p = p->next;
```

At every step of the traversal, two fields are accessed: `key` at offset 0, and `next` at offset 808. The field `key` sits in **cache line 0** of the node (offsets 0–63). The field `next` sits in **cache line 12** (the cache line beginning at offset ⌊808/64⌋ × 64 = 768, covering offsets 768–831). Both are *cache misses* in the general case, because the nodes were allocated individually with `calloc` and are scattered across the virtual address space; there is no spatial or temporal reason for a previously-loaded node's data to be sitting in the cache when we arrive at it.

The immediate fix is just to move the fields around:

```
typedef struct node_t {
    double         key;         //  8 bytes, offset   0
    struct node_t *next;        //  8 bytes, offset   8
    double         data[100];   // 800 bytes, offsets 16 - 815
} node;
```

Now `key` and `next` are both at offsets 0 and 8 respectively — both within the first 64 bytes, both in cache line 0. One cache miss per step is sufficient for the traversal. The total size of the struct is the same but the number of cache line fetches per traversal step is half than before.

---

3. Going further: splitting hot and cold data into separate allocations

The reordering is a good move, but the “cold” data — the 800-byte `data` array — is still physically inside the node struct. During traversal we do not look at it, but it is still there, occupying 12 of the 13 cache lines of each node, potentially evicting other useful data from cache.This is the problem hot and cold field separation is designed to fix.
Again: a field is *hot* if it is accessed frequently during the algorithm's critical path; it is *cold* if it is accessed rarely, or only once a successful result has been found. In our linked-list search example:

- `key` — accessed at every node during traversal → **hot**
- `next` — accessed at every node during traversal → **hot**
- `data` — accessed only once, when the target node is found → **cold**

The general principle is: **keep hot fields spatially close together** so they end up in the same cache line, and separate cold fields so they do not contaminate the cache working set of the hot path.

The more significant move is to take the cold data out entirely and replace it with a pointer:

```
typedef struct node_t {
    double         key;    //  8 bytes
    struct node_t *next;   //  8 bytes
    double        *data;   //  8 bytes (pointer, not data)
} node;                    // 24 bytes total
```

with the actual payloads allocated as one contiguous block:

```
double *alldata = calloc( N * DATASIZE, sizeof(double) );
// later: nodes[nn].data = alldata + nn * DATASIZE;
```

Now the hot part of each node — everything needed for traversal — is 24 bytes. Three hot records fit in a single 64-byte cache line. For N = 100,000 nodes, the hot working set is:

```
100,000 × 24 bytes = 2.4 MB
```

versus 78 MB before; a ~30× reduction. The cold data — `alldata` — is still there, but it is only touched when a search succeeds, at which point one pointer dereference fetches what we need. That one miss we would have paid in any design anyway.

---

4. stop-by: why the time-to-soltion does not reveal things completely, and the role of OoO in establishing a “memory parallelism”

I used to gloss over this, but I think it is better to illustrate the point.

If you benchmark the original layout (`key + data[100] + next`) against the reordered layout (`key + next + data[100]`), measuring wall-clock time on the pointer-chasing traversal, you will almost certainly see no significant, or minor, difference. The struct-split version might show some improvement, but much less than a naive reading of "30× less memory traffic" would make you hope for.
This is not measurement noise, and it does not mean the optimisation is wrong. It means the thing we have optimised is not the thing that dominates the runtime. Sad but true.

What actually happens is the following: when the CPU is at node `p` and needs to evaluate `p->key != key` and then follow `p->next`, both addresses are available as soon as the pointer `p` is known. The out-of-order execution engine will dispatch **both loads simultaneously**; there is no reason to wait for `key` before issuing the request for `next`. The effective latency per traversal step is therefore:

```
max( latency(CL_key), latency(CL_next) ) ≈ DRAM latency
```

not the sum. Putting both fields in the same cache line reduces the number of outstanding requests from 2 to 1, which is good for bandwidth, but it does not change the critical-path latency, since that was already just one DRAM round-trip — the maximum of two parallel fetches is the same as one fetch when both are cache misses.

This is the nature of *pointer-chasing*. Each traversal step depends on the result of the previous one: you cannot know the address of node `p+1` until you have loaded `p->next`. There is no way to overlap the latency of step `p` with step `p+1`, no way for the prefetcher to get ahead, because the next address is not known until the current load completes. The serial dependency chain runs through the entire traversal, every step takes one full DRAM round trip, and field reordering does not break that chain.

Field reordering actually improve the bandwidth.
The original layout consumes 128 bytes of memory bandwidth per traversal step (two cache lines) to fetch 16 bytes of useful data, while int the reordered layout we consume 64 bytes for the same 16 bytes — a 12.5% agaiinst 25%.
But the serial dependency chain is still there, and is the wall-clock time of a latency-bound traversal.

Then, **wall-clock timing is not the best metric here**. We need to look at some hardware counters:

- `PAPI_L3_LDM` will show roughly 2× as many LLC load misses in the original layout as in the reordered one — even if the timings are identical. This is the correct signal that the optimisation is working.
- `PAPI_RES_STL` (cycles stalled on any resource) will be very close to `PAPI_TOT_CYC` in both cases; the traversal loop is almost entirely stall cycles waiting for DRAM.
- The IPC metric (Instructions Per Cycle), derived as `PAPI_TOT_INS / PAPI_TOT_CYC`, will be somewhere around 0.1–0.3 — a striking number (in the bad sense; this figure is awful on a CPU capable of ~4 IPC) for a loop that does almost nothing except a comparison and a pointer dereference.
  When you encounter figures like this, stop and re-consider what you're doing.

---

5. Eliminating pointer chasing: now we see a larger improvement in time

To actually *see* the hot/cold benefit in wall-clock numbers, we need to eliminate the serial dependency chain. The straightforward way is to replace the linked list with a contiguous array:

```
// hotcold_a: array-of-structs, cold layout
typedef struct node_t {
    double key;
    double data[DATASIZE];
} node;
node *nodes = calloc( N, sizeof(node) );

// search:
int nn = 0;
while ( (nn < N) && (nodes[nn].key != key) )
    nn++;
```

```
// hotcold_b: array-of-structs, hot/cold split
typedef struct node_t {
    double  key;
    double *data;
} node;
node   *nodes   = calloc( N,            sizeof(node)   );
double *alldata = calloc( N * DATASIZE, sizeof(double) );
// nodes[nn].data = alldata + nn * DATASIZE;
```

In the array version, `nodes[nn+1]` is at a fixed offset from `nodes[nn]` — the access pattern is perfectly sequential, which the hardware prefetcher handles effortlessly. More importantly, the dependency from one iteration to the next is a simple integer increment, not a memory load; the serial latency chain is gone. Both of these things matter.

The working sets are (for the default value $N = 100,000$):

```
hotcold_a:  N × 808 bytes ≈ 77 MB     (too large for any L3)
hotcold_b:  N ×  16 bytes ≈  1.6 MB   (fits comfortably in L3)
```

For `hotcold_a`, even with sequential access and a working prefetcher, the 77 MB footprint means constant L3 thrashing: each node fetch brings in 12 cache lines of `data` payload that will never be looked at during the search and will be evicted almost immediately. For `hotcold_b`, after the first traversal the entire 1.6 MB hot array is warm in L3, and all subsequent searches run from cache without touching DRAM. The timing difference is large and reproducible — easily an order of magnitude for large N, depending on the L3 size of the machine.

Let's state more clearly what happens:

1. In the pointer-chasing experiments, the field reordering improves bandwidth but not latency, and timing does not reflect the improvement.
2. In the array experiments, the hot/cold split reduces the working set enough that it fits in L3 (for the default N value), which means that the prefetcher can satisfy requests from cache rather than DRAM; timing reflects this immediately.

Code	Pointer chasing?	Timing benefit visible?	What actually changes
`pointer_chasing/hotcold_a`	yes	no	baseline
`pointer_chasing/hotcold_b`	yes	marginal	~2× fewer LLC misses; same serial latency per step
`not_pointer_chasing/hotcold_a`	no	partial	prefetcher helps; but large N does not fit in L3
`not_pointer_chasing/hotcold_b`	no	yes, clearly	reduced working set fits in L3 for larger N; searches run from cache

---

9. Wrap-up

We have discussed three progressively more aggressive steps:

1. **Reorder fields within the struct** so that those accessed together on the critical traversal path fall within the same 64-byte cache line. This halves (or better) the number of cache line fetches per step, which reduces bandwidth consumption. In a pointer-chasing scenario, timing will likely not reflect this; hardware counters will.
2. **Split the struct** so that hot metadata and cold payload live in separate memory regions. The hot working set shrinks dramatically — 32× in our example — which eventually allows it to fit in L3 and eliminates most LLC misses in the non-pointer-chasing case.
3. **Recognise that pointer chasing is a specific problem** that must be addressed separately. It does not matter how well the hot fields are packed if every traversal step is serialised on a DRAM round-trip

waiting for the next pointer. Switching to a contiguous array, or using explicit software prefetching with `__builtin_prefetch`, is what actually breaks the latency chain.

Hardware performance counters (`PAPI_L3_LDM`, `PAPI_RES_STL`, `PAPI_TLB_DM`, IPC derived from `PAPI_TOT_INS / PAPI_TOT_CYC`) are the tool to understand what is happening at each step. Timing alone can be misleading in the pointer-chasing scenario, where two layouts with very different memory traffic profiles may exhibit nearly identical runtimes.

The broader lesson is perhaps this: the cost model for memory-bound code is not `time ∝ bytes_accessed`. It is `time ∝ latency_of_serial_dependency_chain + bandwidth_pressure_on_the_non-serial_parts`.

Separating these two contributions, understanding which dominates in a given piece of code, and using hardware counters to verify your model — that is what it means to do cache optimisation.

Appendix: sizes and working sets

Code	Struct size	Hot working set (N=100K)	Cache lines per traversal step
`pointer_chasing/hotcold_a` (`key + data[100] + next`)	816 B	78 MB	2 (CL 0 and CL 12)
`pointer_chasing/hotcold_b` (`key + next + data*`)	24 B	2.4 MB	1 (CL 0 contains all hot fields)
`not_pointer_chasing/hotcold_a` (`key + data[100]`, array)	808 B	77 MB	sequential, prefetched; working set too large for L3
`not_pointer_chasing/hotcold_b` (`key + data*`, array)	16 B	1.6 MB	≈0 LLC misses after warmup; L3-resident

Cache line: 64 bytes. Natural alignment assumed throughout (all fields 8-byte aligned, no compiler padding added). N = 100,000, DATASIZE = 100 doubles.